

V.A.P. (Vibe Audio Protocol) Standard

"The 9-Dimensional Holographic Audio Standard"

1. Executive Summary & Project Context

The Vibe Audio Protocol (V.A.P.) replaces outdated ID3 tags by capturing the structural, emotional, and physical impact of audio. It defines a 9-pillar taxonomy (Structural, Tonal, Timbral, Linguistic, Affective, Contextual, Photometric, Kinetic, Genealogical). This repository houses the official JSON schemas, the technical manual, and the DSP/ML extraction engine required to analyze raw `.flac` or `.wav` files and output standardized V.A.P. metadata for use in SoulSync.

2. Technical Specifications

- **Data Standard:** JSON Schema Draft-07 (Strict Validation).
- **Extraction Engine:** Python (Librosa for DSP, HuggingFace Transformers for Linguistic/Affective NLP).
- **Speed:** Rust-based high-speed JSON parser for real-time playback integration.
- **Output:** `.vap.json` sidecar files or embedded metadata streams.

3. Repository Structure

```

aurphyx-vap/
├── .github/
│   └── workflows/
│       └── schema_validation.yml
├── schema/
│   ├── vap_schema_v3_1.json      # The official JSON Schema definition
│   └── examples/                 # Example payloads (Celine Dion, Stuca)
├── engine/                       # The V.A.P. Scoring & Extraction Engine
│   ├── pyproject.toml
│   ├── src/
│   │   ├── dsp_analyzer.py      # Structural, Tonal, Timbral extraction
│   │   ├── nlp_analyzer.py      # Linguistic, Affective extraction
│   │   ├── context_engine.py    # Generates Contextual, Photometric, Kinetic
│   │   └── master_generator.py  # Orchestrates and outputs the JSON
│   └── tests/
│       └── test_engine.py
├── parser-rs/                   # High-speed Rust parser for clients
│   ├── Cargo.toml
│   └── src/lib.rs
└── VAP_Technical_Manual.md      # Full documentation of the 9 pillars

```

└─ README.md

4. CI/CD Workflow (.github/workflows/schema_validation.yml)

```
name: VAP Standard & Engine CI
on:
  push:
    branches: [ "main" ]

jobs:
  validate-schema:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Validate JSON Schema
        uses: nhurden/ajv-action@v3
        with:
          schema: schema/vap_schema_v3_1.json
          data: schema/examples/*.json

  test-extraction-engine:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'
      - name: Install Dependencies
        working-directory: ./engine
        run: |
          pip install -r requirements.txt
          pip install pytest
      - name: Run Engine Tests
        working-directory: ./engine
        run: pytest tests/

  build-rust-parser:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
      - name: Test Rust Parser
        working-directory: ./parser-rs
        run: cargo test
```

5. Build & Deployment Guide

Using the Schema

The JSON schema is statically hosted. To validate data in any language, point your schema validator to:

https://raw.githubusercontent.com/aurphyx/vap/main/schema/vap_schema_v3_1.json

Running the V.A.P. Scoring Engine (Local Analysis)

1. Setup Python Environment:

```
cd engine
python -m venv venv
source venv/bin/activate
pip install -e .
```

2. Analyze an Audio File:

```
vap-analyze --input /path/to/song.flac --output /path/to/output.vap.json
```

Building the High-Speed Rust Parser (For OS/Player Integration)

```
cd parser-rs
cargo build --release
# Generates libvapparse.so / .dll for embedding into AuraOS or SoulSync
```